

Agentes Cooperativos (Cooperative Agents) — Análisis completo de la Sesión 11

Fuente base: *Agentes IA — Cooperative Agents* — Módulo 4 (Agentes Cognitivos), Programa en Diseño e Implementación de Agentes IA, UTEC Posgrado. Dictada por MEng. Boris Alzamora.

Complementado con: investigación propia sobre el estándar **A2A** (*Agent-to-Agent*, "Agente a Agente"), su comparación con **MCP** (*Model Context Protocol*, "Protocolo de Contexto de Modelo"), la especificación pública del proyecto `a2aproject/A2A`, y los patrones clásicos de arquitectura multiagente (orquestador/supervisor, red/malla, jerárquico) usados en frameworks como LangGraph, AutoGen y CrewAI.

Propósito de este documento: las Sesiones 8, 9 y 10 construyeron progresivamente **un** agente: sus componentes (Sesión 8), su memoria (Sesión 9) y su programa de decisión (Sesión 10). Esta sesión da el salto de "un agente" a "varios agentes que colaboran": primero de la forma más simple posible (archivos compartidos en disco), y luego mediante un estándar abierto de interoperabilidad, **A2A**.

0. Dónde se ubica esta sesión — de la decisión individual a la colaboración

Sesión 8 – Arquitectura de Agentes:

Agente = comunicación + contexto + entorno + autonomía + criticidad
└ Se define QUÉ componentes tiene UN agente.

Sesión 9 – Memoria Contextual:

Context Window + Short/Long Term Memory + Store/Checkpoint
└ Se define QUÉ recuerda UN agente.

Sesión 10 – Agentes Reflexivos:

Percepto → Estado interno → Meta/Utilidad/Aprendizaje → Acción
└ Se define CÓMO decide UN agente.

Sesión 11 – Agentes Cooperativos:

Agente A —(archivo | protocolo)→ Agente B
└ Se define CÓMO colaboran VARIOS agentes que no comparten ni memoria ni herramientas entre sí.

El curso presenta la colaboración multiagente en dos niveles de madurez, en ese orden:

1. **Colaboración por medios externos** (Bloque A): dos agentes independientes que se pasan información a través de un archivo en disco. No hay protocolo, no hay descubrimiento de capacidades, no hay estándar: solo un contrato implícito sobre el formato del archivo.
2. **A2A** (Bloque B): el mismo problema — agentes que no comparten memoria ni herramientas — resuelto con un estándar abierto, con descubrimiento de capacidades ("Agent Cards"), transporte definido (HTTP, JSON-RPC) y ciclo de vida de tareas.

La idea pedagógica central es que **A2A no es un concepto nuevo, es una formalización** de algo que ya se puede hacer con un archivo de texto: agentes que colaboran sin fusionar su estado interno.

1. Objetivos y agenda de la sesión

Objetivos declarados en el PDF:

1. Revisión de colaboración por medios externos (variables, archivos).
2. **A2A**, extendiendo la colaboración entre agentes.

Agenda del Bloque A:

#	Tema	Qué se trabaja
1	Implementando un agente en LangChain a <i>files</i>	Un agente que persiste su salida a un archivo
2	Implementando un segundo agente que colabora por archivos	Un segundo agente que lee ese archivo y continúa el trabajo
3	Lab: aterrizando a proyectos	Check de configuración de ambiente por grupos (Python, VS Code, Ollama / OpenAI / GoogleAI / otro) y revisión de un trabajo en VS Code

Agenda del Bloque B:

#	Tema	Qué se trabaja
1	A2A	Qué es, características, aplicaciones, principios de diseño
2	Lab: aterrizando a proyectos	Reflexión e implementación en Python

Tarea final indicada en la página 16 del PDF:

Analizar y sustentar la aplicabilidad (o no aplicabilidad) de agentes colaborativos en el proyecto. Proponer un esquema colaborativo de agentes por medios externos y su posible extensión a A2A. **Fecha límite: 25/07.**

2. Ejercicio guiado — el ejemplo completo de *Agentic System Profile Card* (página 2)

Antes de entrar al tema de la sesión, el PDF abre con un ejemplo ya resuelto y completo del **Agentic System Profile Card** (la plantilla de documentación de agentes que el curso viene pidiendo actualizar sesión a sesión: Sesión 9 con memoria, Sesión 10 con tipo de agente reflexivo). Aquí se muestra el caso de un agente de **reclutamiento de RRHH** (*Recursos Humanos*) totalmente diligenciado, que sirve como referencia de qué tan detallada debe quedar la ficha de un agente real.

2.1 Communication Layer

Communication Layer: No Conversacional

El agente no interactúa con un usuario final por chat; opera como un servicio de *back-office* que recibe currículums y entrega una lista priorizada.

2.2 Context Definition

Bloque	Contenido
Domain Definition	Recursos Humanos, especialista en Adquisición de Talento. Proceso de selección y filtrado inicial de candidatos para una vacante laboral específica.
Objectives Definition	Analizar un gran volumen de currículums (CV, <i>Curriculum Vitae</i>) de manera automática. Extraer y estandarizar información relevante (habilidades, experiencia, educación). Calificar y clasificar a los candidatos según su grado de afinidad con los requisitos del puesto. Entregar una lista priorizada (" <i>shortlist</i> ") al reclutador para optimizar su tiempo.

2.3 Environment Definition

Bloque	Contenido
Knowledge	MOF (<i>Manual de Organización y Funciones</i>): documento principal que define los criterios de éxito del puesto. Vectorial DB / Docs: base de datos de currículums de los postulantes, base de datos de conocimientos de RRHH (ontologías de habilidades que relacionan, p. ej., "Project Management" con "PMP" o "Scrum Master"). Políticas de contratación y guías de la empresa. Historial de contrataciones pasadas para aprender patrones de éxito.
Tools	Conexión con Sistemas de Seguimiento de Candidatos (ATS , <i>Applicant Tracking System</i>) para obtener vacantes y enviar resultados. Conexión con portales de empleo (LinkedIn) para la ingesta de perfiles. Modelo analítico: motor de PLN (<i>Procesamiento del Lenguaje Natural</i>) para el parseo y extracción de entidades de los CV. Algoritmo de <i>scoring</i> : un modelo de clasificación o un sistema de reglas ponderadas (personalizable por el reclutador) para calcular la puntuación de afinidad.
Short Term Memory	Los datos de la vacante activa y los CV que se están procesando en la sesión actual.
Long Term Memory	Preferencias del reclutador (p. ej., "dar más peso a la experiencia"). Perfiles de candidatos de procesos anteriores para futuras búsquedas. <i>Feedback</i> del reclutador sobre la calidad de las clasificaciones para el reentrenamiento del modelo.

2.4 Autonomy Dimension Definition

Semi-Autónomo | Constreñido (Constrained)

El agente automatiza completamente el análisis y la clasificación, pero **no toma la decisión final de contratación**: su función es asistir y recomendar. Su autonomía está limitada estrictamente a las reglas y requisitos definidos en la descripción del puesto y por el reclutador. No puede "improvisar" criterios.

Esto reutiliza directamente el vocabulario de la Sesión 8 (*Constrained* → *Semi-Autonomous* → *Fully Autonomous*): aquí el agente vive justo en el límite entre ambos niveles, con la decisión final siempre humana.

2.5 Criticality Dimension Definition

Nivel de Criticidad: Medio-Alto (Controlado)

Riesgos (Risk) identificados en la ficha:

Riesgo	Descripción
Sesgo algorítmico (<i>bias</i>)	El mayor riesgo: el agente podría aprender sesgos inconscientes presentes en los datos históricos de contratación (favorecer candidatos de ciertas universidades, género o grupos demográficos), perpetuándolos y contradiciendo el objetivo de promover la diversidad.
Mala interpretación de CV	El PLN puede fallar al interpretar formatos de CV no convencionales, jerga específica o habilidades descritas de forma creativa, descartando injustamente a un candidato válido.
Sobreoptimización a palabras clave (<i>overfitting</i>)	El modelo podría volverse demasiado rígido y buscar coincidencias exactas con las palabras de la descripción del puesto, ignorando candidatos con experiencia equivalente descrita con sinónimos.
Datos personales	Un manejo inadecuado, una brecha de seguridad o un uso no autorizado de esta información puede acarrear consecuencias legales (infringiendo leyes como la Ley de Protección de Datos Personales) y un daño reputacional severo.

Guardrails, Evals y LLM-as-a-Judge (*Guardrails, Evals, LLMaaJ and Controls*):

Control	Descripción
Transparencia y explicabilidad (XAI , <i>Explainable AI</i>)	El sistema DEBE justificar por qué un candidato recibió una puntuación alta o baja (p. ej., "puntuación alta por 5 años de experiencia en Python y certificación PMP"). Esto permite al reclutador entender y validar (o refutar) la recomendación del agente.
Supervisión humana (HITL , <i>Human-in-the-loop</i>)	El reclutador siempre tiene la última palabra. La interfaz debe facilitar la revisión de todos los perfiles, incluidos los de baja puntuación, y permitir que el humano corrija la clasificación del agente.
Auditoría de sesgos y métricas de equidad	Implementar un módulo de auditoría que analice las recomendaciones del agente en busca de patrones sesgados. Este " <i>Guardian Agent</i> " o " <i>Judge</i> " puede alertar si, por ejemplo, candidatos mejor clasificados provienen desproporcionadamente de un solo género o universidad.
<i>Feedback</i> continuo	Crear un mecanismo para que los reclutadores califiquen la calidad de las recomendaciones. Este <i>feedback</i> es crucial para reentrenar y afinar el modelo continuamente, corrigiendo errores y sesgos a lo largo del tiempo.

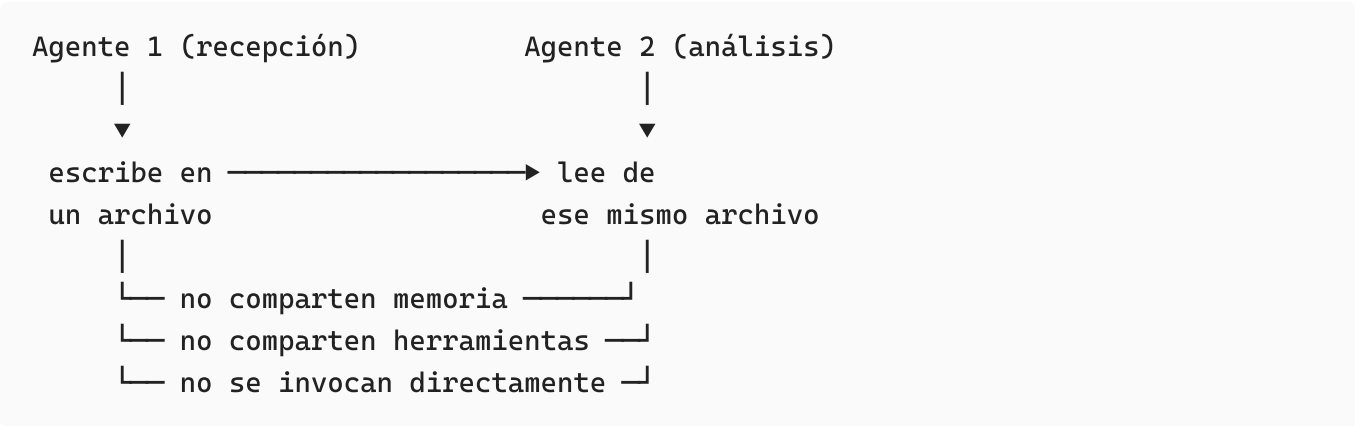
Control	Descripción
Anonimización de perfiles	Anonimizar los datos privados que figuren en el CV con un componente local previo a subirlo al agente.

Por qué importa este ejemplo en la Sesión 11: el *"Guardian Agent"/"Judge"* mencionado en Auditoría de Sesgos es, en sí mismo, un segundo agente que colabora con el agente principal de clasificación — exactamente el patrón que la sesión va a formalizar a continuación. La ficha ya venía anticipando, sin nombrarlo, un esquema de agentes cooperativos: uno que clasifica y otro que audita.

3. Bloque A — Colaboración por medios externos (archivos)

3.1 La idea central

El PDF no dedica una diapositiva teórica a este bloque: lo enseña directamente con dos ejercicios de laboratorio (*hands-on*) de 20 minutos cada uno. La lección se extrae de la arquitectura resultante, no de una definición explícita:



Dos procesos Python completamente independientes, cada uno con su propio LLM (*Large Language Model*, modelo de lenguaje de gran escala), sus propias *tools* y su propio ciclo de vida, coordinados únicamente por un contrato de formato sobre un archivo de texto plano. No hay A2A, no hay MCP (*Model Context Protocol*), no hay ni siquiera una cola de mensajes: el "protocolo" es el formato del archivo `reclamos_registrados.txt`.

3.2 Enunciado del Laboratorio 1 (página 6)

Elabore un agente que se encargue de la recepción de reclamos de un banco por medio de chat y que persista el historial a un archivo. — **20 min**, con LangChain + Ollama.

3.3 Enunciado del Laboratorio 2 (página 7)

Elabore un segundo agente que se encargue del análisis del reclamo, y de determinar una resolución final al mismo. Como resultado, debe entregar un reporte en Markdown. — **20 min**, con LangChain + Ollama.

3.4 Implementación — agente_reclamos_bancarios.py (Agente 1: recepción)

Archivo real de la carpeta de la sesión. Construido con `langchain.agents.create_agent` (LangChain 1.x) y el modelo `anthropic:claude-sonnet-4-6`.

Rol: agente conversacional de atención al cliente de un banco, especializado en reclamos de tarjeta de crédito.

Memoria de corto plazo — archivo plano, no Store de LangGraph:

```
ARCHIVO_HISTORIAL = Path("historial_chat.txt")
MAX_MENSAJES_MEMORIA = 10 # 5 turnos de usuario + 5 de agente
```

Cada línea de `historial_chat.txt` es un objeto JSON independiente (`{"fecha", "role", "content"}`), escrito con `open(..., mode="a")`. Al reiniciar el programa, `cargar_memoria_corta()` relee solo las últimas `MAX_MENSAJES_MEMORIA` líneas. El propio código comenta un detalle importante de diseño:

El límite de mensajes solo aplica al *recuperar* historial tras reiniciar el programa. No debe aplicarse dentro de una conversación en curso, o el agente "olvida" datos ya proporcionados a mitad de tarea.

Esto es una variante deliberadamente simplificada de la estrategia `trim_count` que la Sesión 9 ya había formalizado (recortar el estado por cantidad de mensajes), pero aplicada solo en el borde de arranque del proceso, no en cada turno.

Herramienta de escritura — el contrato de formato:

```
@tool
def registrar_reclamo(nombre_cliente, dni_cliente, ultimos_cuatro_digitos,
                      fecha_operacion, monto, comercio, descripcion,
                      solucion_solicitada) -> str:
    ...
    codigo_reclamo = f"REC-{uuid4().hex[:8].upper()}"
    reclamo = f"""
=====
CÓDIGO DEL RECLAMO: {codigo_reclamo}
FECHA DE REGISTRO: {fecha_registro}
PRODUCTO: Tarjeta de crédito
```

```

CLIENTE: {nombre_cliente}
DNI: {dni_cliente}
...
ESTADO: Registrado
=====
"""
    ARCHIVO_RECLAMOS.open(mode="a", ...).write(reclamo)

```

Este bloque de texto con formato `CLAVE: valor` delimitado por una línea de `=` es, literalmente, el "protocolo de interoperabilidad" entre los dos agentes. No existe *schema* JSON, no hay validación de tipos entre procesos: el segundo agente debe **volver a parsear texto** para entender el reclamo.

Validaciones de negocio antes de escribir (el agente no confía ciegamente en el LLM):

Campo	Validación
<code>dni_cliente</code>	Exactamente 8 dígitos numéricos
<code>ultimos_cuatro_digitos</code>	Exactamente 4 dígitos numéricos
<code>monto</code>	Mayor que cero

Herramientas de consulta (`consultar_reclamo_por_codigo`, `consultar_reclamos_por_dni`, `consultar_reclamos_por_nombre`): reconstruyen los bloques del archivo con `_extraer_bloques_reclamos()` y `_parsear_bloque_reclamo()`, separando por la línea de `=` y buscando prefijos de campo. El *system prompt* le indica al LLM un orden de preferencia explícito: código > DNI > nombre, porque el nombre no es un identificador único (puede haber homónimos).

Reglas de negocio en el *system prompt*, no en código:

- Nunca pedir el número completo de tarjeta, contraseña, PIN, CVV ni códigos SMS.
- Presentar un resumen del reclamo y pedir confirmación explícita antes de llamar a `registrar_reclamo`.
- Si es un consumo no reconocido, recomendar contactar canales oficiales para bloquear la tarjeta — **sin afirmar que el agente mismo la bloqueó** (no tiene esa capacidad; es una restricción explícita contra alucinación de capacidades).

3.5 Implementación — `agente_analista_reclamos.py` (Agente 2: análisis y resolución)

El propio archivo se autodocumenta como el segundo eslabón del flujo:

Este es el SEGUNDO agente del flujo. El primero (agente_reclamos_bancarios.py) recibe al cliente y registra el reclamo en ARCHIVO_RECLAMOS . Este agente toma esos reclamos ya registrados, los analiza contra reglas de negocio y determina una resolución final, entregando un reporte en Markdown. No conversa con el cliente: se opera dando el código de un reclamo ya existente.

Decisión de diseño deliberada — duplicación de código, no importación:

```
# El formato de bloques (separador, campos "CLAVE: valor") es el
# mismo que escribe registrar_reclamo() en agente_reclamos_bancarios.py.
# Se reimplementa aquí en vez de importar ese módulo para que este
# archivo funcione como un agente independiente.
```

_extraer_bloques_reclamos() y _parsear_bloque_reclamo() están **copiadas letra por letra** del primer agente, en vez de compartir un módulo común. Esto no es un descuido: es la demostración práctica del principio de diseño que la sesión atribuirá luego a A2A — *"agentes colaboran sin compartir memoria ni herramientas"*. Aquí ni siquiera comparten código de parseo por importación; solo comparten un contrato de formato de texto.

Reglas de negocio para decidir la resolución (codificadas en el *system prompt*, con un umbral parametrizado en Python):

```
UMBRAL_ESCALAMIENTO_FRAUDE = 3000.00
```

Tipo de reclamo	Regla de resolución
Consumo no reconocido, monto < umbral	Aprobado — reembolso total
Consumo no reconocido, monto ≥ umbral	Escalado a investigación de fraude (nunca aprobación directa)
Cobro duplicado, con evidencia clara en el detalle	Aprobado — reembolso total del monto duplicado, sin importar el monto
Monto incorrecto, con monto correcto indicado	Aprobado — reembolso parcial por la diferencia
Monto incorrecto, sin monto correcto indicado	Pendiente de información adicional
Comisión/pago no reflejado, sin evidencia suficiente	Pendiente de información adicional

Herramienta `generar_reporte_resolucion` : genera el `.md` en `reportes_reclamos/{codigo}.md` con una tabla de datos del reclamo, la decisión, el monto a reembolsar, la justificación y los siguientes pasos — y además **reescribe** la línea `ESTADO:` del bloque original en `reclamos_registrados.txt` (vía `_actualizar_estado_reclamo`), cerrando el ciclo de vida del reclamo sobre el mismo archivo que usa el primer agente.

Ejemplo real generado por el laboratorio (`reportes_reclamos/REC-F5898854.md`): un reclamo de S/ 1500.00 por un consumo no reconocido en LATAM AIRLINES, resuelto como "Aprobado - Reembolso total" con justificación explícita ("el monto reclamado es menor al umbral de S/ 3,000.00").

3.6 Qué enseña esta pareja de agentes

Dimensión	Lo que muestra
Acoplamiento	Ninguno en tiempo de ejecución: los procesos ni siquiera necesitan correr al mismo tiempo
Medio de coordinación	Un archivo de texto plano con formato fijo
Fragilidad del contrato	Si <code>registrar_reclamo</code> cambia una etiqueta de campo, <code>agente_analista_reclamos.py</code> deja de poder parsear reclamos nuevos sin que nada lo avise en tiempo de compilación
Descubrimiento de capacidades	Inexistente: el segundo agente "sabe" de antemano qué <i>tools</i> tiene el primero porque un humano leyó el código fuente
Ventaja	Extremadamente simple, cero dependencias de infraestructura, fácil de depurar leyendo el archivo a ojo
Límite	No escala a más de un par de agentes ni a formatos que cambian con frecuencia; no hay autenticación ni control de acceso por agente

Esta tabla es exactamente el punto de partida que A2A busca resolver en la segunda mitad de la sesión.

3.7 Contexto adicional en el repositorio — variantes ya exploradas en la Sesión 9

La carpeta de la Sesión 11 también contiene `main.py` y `agente_langchain_reclamos.py`, que **no** corresponden al ejercicio de dos agentes por archivo, sino que son continuaciones directas del agente de memoria de la Sesión 9 (`agente_landgraph_memoria_corto_largo_plazo.py`) adaptado a este mismo dominio de reclamos bancarios. Se documentan aquí porque conviven en la misma carpeta y ayudan a contrastar dos formas de persistir memoria:

Archivo	Framework	Corto plazo
main.py	LangGraph (StateGraph manual)	SqliteSaver → historial_chat.sql estrategias de la Sesión trim_count , trim_summary)
agente_langchain_reclamos.py	langchain.agents.create_agent	SqliteSaver → historial_chat_langchain

Observación arquitectónica clave del propio código: `create_agent` no permite reconstruir el *system prompt* turno a turno con datos ya conocidos, como sí hacía el nodo manual de `main.py`. Por eso, en `agente_langchain_reclamos.py`, la memoria de largo plazo (reclamos previos del cliente) se expone como una **tool** (`consultar_reclamos_previos`) que el agente decide invocar, en vez de inyectarse a la fuerza en el prompt. El propio archivo lo resume así:

Es el patrón idiomático de LangChain: si el agente necesita un dato externo, se le da una *tool* para pedirlo.

Esto es relevante para la Sesión 11 porque es la misma lógica que después justificará por qué A2A expone capacidades mediante **Agent Cards** en vez de compartir memoria directamente: si un agente necesita algo de otro, se le da un mecanismo formal para pedirlo, no acceso directo a su estado interno.

4. Bloque B — Agent-to-Agent (A2A)

4.1 ¿Qué es A2A? (página 9 del PDF)

Un estándar abierto diseñado para permitir la comunicación, colaboración y coordinación entre agentes de inteligencia artificial, incluso si fueron desarrollados por distintos proveedores o *frameworks*.

Características clave listadas en el PDF:

Característica	Descripción
Interoperabilidad universal	Agentes de distintos sistemas pueden interactuar sin fricciones

Característica	Descripción
Descubrimiento de capacidades	Uso de "Tarjetas de Agente" (<i>Agent Cards</i>) para conocer habilidades entre agentes
Gestión de tareas	Ciclo de vida estandarizado con estados y transiciones claras
Multimodalidad	Soporta texto, audio, video y datos estructurados
Actualizaciones en tiempo real	Ideal para tareas largas y colaboración continua
Seguridad empresarial	Autenticación y autorización integradas

El diagrama de la página 9 muestra el flujo básico: un **Client Agent** envía una tarea; un **Remote Agent** la ejecuta y devuelve artefactos (representados como una tabla ✓ y un documento ✓/X); y el ciclo completo descansa sobre cuatro pilares visuales: *Secure Collaboration*, *Task and State Management*, *User Experience Negotiation* y *Capability Discovery*.

4.2 Aplicaciones y principios de diseño (página 10)

Aplicaciones:

- Automatización de procesos complejos (RRHH, atención al cliente, logística).
- Coordinación entre agentes especializados (búsqueda, ejecución, comunicación).
- Integración de soluciones de IA en entornos empresariales heterogéneos.

Principios de diseño:

Basado en estándares abiertos: HTTP, JSON-RPC, SSE.
Seguro por defecto: OpenAPI y autenticación robusta.
Agentes colaboran sin compartir memoria ni herramientas.

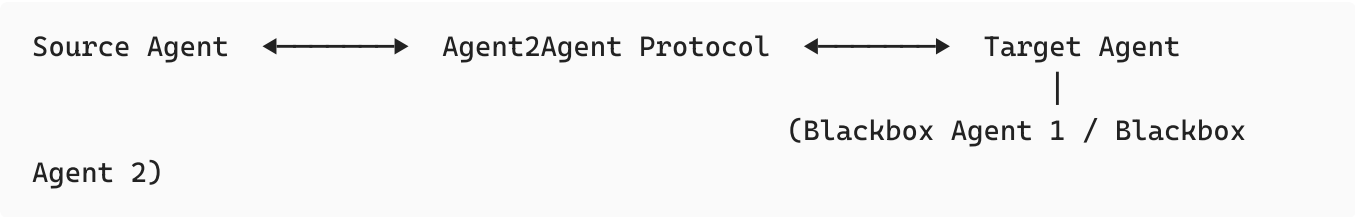
Aquí aparece explícito el principio que ya se había visto **implementado sin nombrarse** en el Bloque A: el agente analista de reclamos no comparte memoria (no lee el `Store` del agente de recepción) ni herramientas (reimplementa su propio *parsing*) con el agente de recepción. A2A simplemente pone nombre y estándar a ese aislamiento deliberado.

Expansión de las siglas del principio de diseño:

Sigla	Nombre completo	Rol en A2A
HTTP	<i>HyperText Transfer Protocol</i>	Transporte de las peticiones entre agentes
JSON-RPC	<i>JavaScript Object Notation – Remote Procedure Call</i>	Formato de los mensajes de invocación (JSON-RPC 2.0)

Sigla	Nombre completo	Rol en A2A
SSE	<i>Server-Sent Events</i>	Transporte para actualizaciones en tiempo real durante tareas largas
OpenAPI	Especificación abierta para describir APIs REST	Referencia de diseño para exponer capacidades de forma documentada

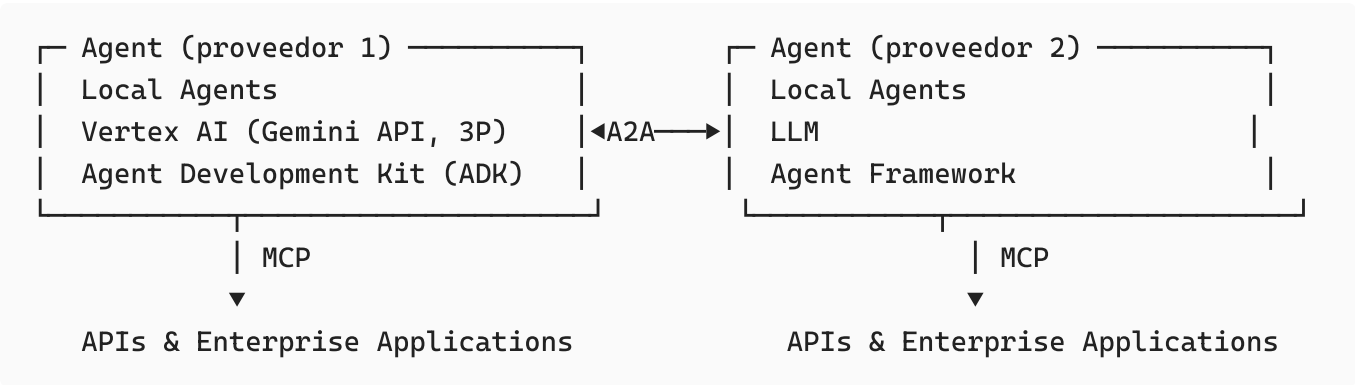
4.3 Diagrama — Agent2Agent Protocol (página 10)



La palabra clave visual del diagrama es que el *Target Agent* aparece explícitamente como una **caja negra** (*blackbox*): el *Source Agent* no necesita conocer su implementación interna, su *framework* ni su modelo — solo su Agent Card.

4.4 Diagrama — Arquitectura ADK + A2A (página 11)

El PDF muestra dos "cajas" de agente equivalentes, cada una desde la perspectiva de un proveedor distinto, comunicadas por A2A a través de un límite organizacional o tecnológico ("*Organizational or technological boundaries*"):



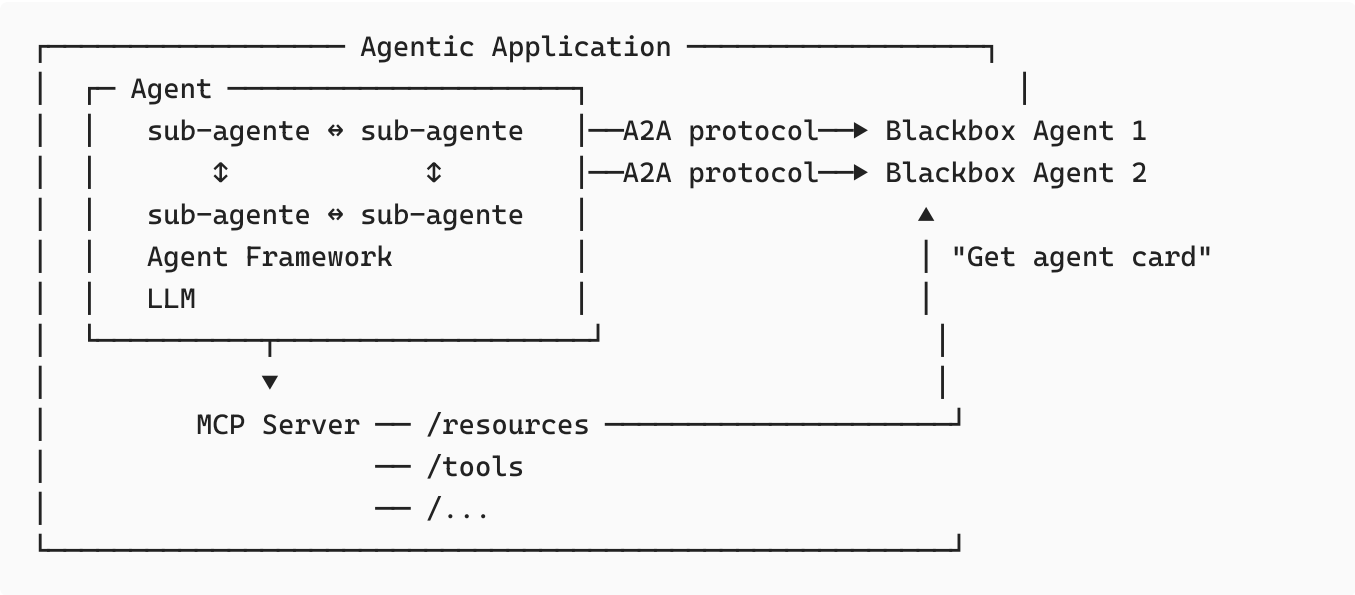
Lectura de este diagrama: A2A y MCP (*Model Context Protocol*) no son competidores, son **ortogonales**:

- **MCP** conecta verticalmente un agente con sus propias herramientas, datos y APIs internas (aquí, "APIs & Enterprise Applications").
- **A2A** conecta horizontalmente un agente con **otro agente**, potencialmente de otro proveedor (Vertex AI/Gemini con **ADK** de un lado, cualquier otro LLM y *framework* del otro).

Expansión de siglas:

Sigla	Nombre completo
ADK	<i>Agent Development Kit</i> — kit de Google para construir agentes
API	<i>Application Programming Interface</i>
MCP	<i>Model Context Protocol</i>
LLM	<i>Large Language Model</i>
3P	<i>Third Party</i> (modelos de terceros servidos vía Vertex AI)

4.5 Diagrama — Aplicación agéntica con sub-agentes, MCP y A2A (página 12)



Este diagrama es el más completo de la sesión porque combina **tres** capas de colaboración en un solo sistema:

1. **Sub-agentes internos** (dentro de la misma aplicación agéntica): coordinación interna, sin necesidad de A2A — es orquestación intra-proceso, del tipo que ya podría implementarse con LangGraph (grafo de nodos-agente) o un patrón supervisor/*sub-agent*.
2. **MCP Server**: el agente principal expone (o consume) recursos y herramientas mediante MCP.
3. **A2A hacia agentes externos** ("*Blackbox Agent 1*", "*Blackbox Agent 2*"): agentes ajenos a la aplicación, de los que solo se conoce su *Agent Card* ("*Get agent card*") — el mecanismo formal de descubrimiento de capacidades.

4.6 Diagrama — Client Agent y múltiples Remote Agents (página 13)

```
User → Client Agent —A2A→ Remote Agent 1 (p. ej., proveedor con logo verde/blanco)
                                —A2A→ Remote Agent 2 (p. ej., barco/velero)
                                —A2A→ Remote Agent 3 (p. ej., estrella/sol)
```

Este es el patrón de **orquestador con agentes remotos heterogéneos**: un único punto de entrada para el usuario (*Client Agent*) que delega tareas específicas a distintos *Remote Agents*, cada uno potencialmente de un proveedor de IA distinto — el ejemplo visual sugiere íconos de distintas marcas para enfatizar que A2A no asume que todos los agentes remotos comparten *stack* tecnológico.

4.7 Ejemplo real de plataforma A2A-ready — Google Agentspace (página 14)

El PDF incluye una captura de **Google Agentspace**, una interfaz donde un usuario ("Hello, Andy.") puede:

- Buscar contenido o hacer preguntas (*Deep research* , *Search* , *Sources*).
- Invocar **Agentes** ya configurados como tarjetas: *Deep Research*, *IdeaForge*, *Data Scientist*, o crear uno nuevo (*Create Agent*).
- Usar **Prompts** predefinidos: *Translate Text*, *Draft Email*, *Generate image*, *Chat with content*.

Este ejemplo aterriza el concepto abstracto de "Agent Card" en una interfaz real: cada tarjeta de agente en Agentspace (Deep Research, IdeaForge, Data Scientist) es, conceptualmente, la superficie visible de algo que — si se expusiera vía A2A — sería descubrible por otro agente cliente sin necesidad de conocer su implementación.

4.8 Referencia de código dada en el PDF (página 15)

```
https://github.com/a2aproject/a2a-python
```

Es el SDK oficial en Python del proyecto A2A, mencionado junto al logo de LangChain como las dos piezas técnicas con las que se espera que el estudiante implemente la extensión A2A de su proyecto final.

5. Investigación complementaria — lo que el PDF no cuenta sobre A2A

5.1 Origen y gobernanza

A2A fue anunciado por **Google Cloud en abril de 2025** como protocolo abierto para interoperabilidad entre agentes de distintos proveedores. En **junio de 2025**, Google donó el protocolo A2A a la **Linux Foundation**, que constituyó el proyecto comunitario "Agent2Agent Project" con el respaldo inicial de **Amazon Web Services, Cisco, Google, Microsoft, Salesforce, SAP y ServiceNow**. El objetivo declarado de la donación es evitar que A2A quede controlado por un solo proveedor y asegurar que su evolución sea gobernada de forma neutral, de forma similar a como Kubernetes pasó de ser un proyecto interno de Google a un proyecto de la Cloud Native Computing Foundation.

5.2 Mecanismo de descubrimiento — el *Agent Card*

Cada agente que implementa A2A publica un documento JSON (**Agent Card**) en una ruta bien conocida:

```
https://<host-del-agente>/.well-known/agent.json
```

siguiendo la convención de **RFC 8615** (*well-known URIs*). El *Agent Card* describe la identidad del agente, sus capacidades ("*skills*"), el punto de acceso del servicio, los modos de entrada/salida soportados y los esquemas de autenticación requeridos. Esto es exactamente lo que el diagrama de la página 12 llama "*Get agent card*": antes de delegar una tarea a un *Blackbox Agent*, el agente cliente primero descarga su tarjeta para saber qué puede pedirle y cómo autenticarse.

5.3 Transporte y ciclo de vida de las tareas

Toda la comunicación ocurre sobre **HTTP(S)** usando **JSON-RPC 2.0** como formato de mensaje, enrutado a través de un único *endpoint*. La unidad fundamental de trabajo es la **Task** (tarea), identificada por un ID único y con un ciclo de vida formal de estados:

```
submitted → working → input-required → completed
                        ↳ failed
                        ↳ canceled
                        ↳ rejected
                        ↳ auth-required
```

Esto formaliza justamente lo que en el Bloque A quedaba implícito en la columna ESTADO: del archivo `reclamos_registrados.txt` (Registrado → Resuelto - <decisión>): en A2A ese ciclo de vida no es una convención de texto libre, sino un conjunto fijo de estados que cualquier cliente A2A sabe interpretar sin leer el código del agente remoto.

5.4 A2A frente a MCP — diferenciar en vez de confundir

Dimensión	MCP (<i>Model Context Protocol</i>)	A2A (<i>Agent-to-Agent</i>)
Creador	Anthropic (noviembre 2024)	Google (abril 2025), donado a Linux Foundation (junio 2025)
Dirección de la conexión	Vertical: modelo/agente → herramientas y datos	Horizontal: agente → agente
Capa que estandariza	Integración de <i>tools</i> /contexto	Coordinación y delegación de tareas entre agentes autónomos
Transporte típico	JSON-RPC 2.0 sobre <i>stdio</i> /SSE	HTTP(S) + JSON-RPC 2.0 + <i>Agent Cards</i>
Analogía	El "USB-C de los agentes" hacia sus herramientas	El "protocolo diplomático" entre agentes soberanos
¿Compiten?	No — son complementarios: un mismo sistema multiagente típicamente usa MCP para que cada agente acceda a sus propios datos/herramientas, y A2A para que esos agentes se coordinen entre sí	

El propio diagrama de la página 11 del PDF (Sección 4.4 de este documento) ya mostraba esta complementariedad: cada "caja" de agente usa MCP hacia abajo (sus APIs) y A2A hacia el costado (el otro agente).

5.5 Patrones clásicos de arquitectura multiagente (más allá de A2A)

A2A resuelve el transporte y el descubrimiento, pero no impone una topología. En la práctica (LangGraph, AutoGen, CrewAI, OpenAI Agents SDK), los patrones más comunes para organizar **varios** agentes son:

Patrón	Cómo funciona	Cuándo se ve en esta sesión
Pipeline / secuencial	Un agente termina su trabajo y el siguiente lo retoma, sin ida y vuelta	El Bloque A completo: Agente de recepción → archivo → Agente analista
Orquestador / supervisor	Un agente central recibe la tarea del usuario y delega partes a agentes	Diagrama de la página 13: <i>Client Agent</i> → <i>Remote Agent 1/2/3</i>

Patrón	Cómo funciona	Cuándo se ve en esta sesión
	especializados, integrando sus respuestas	
Red / malla (swarm)	Los agentes se comunican entre sí sin un coordinador central fijo, transfiriéndose el control dinámicamente	No aparece explícitamente en el PDF, pero es el patrón que MCP+A2A combinados en la página 12 dejan abierto (sub-agentes internos con comunicación bidireccional)
Jerárquico	Un supervisor delega a sub-supervisores, que a su vez delegan a agentes hoja	Insinuado en el diagrama de sub-agentes de la página 12, aunque el PDF no lo desarrolla a fondo

5.6 Riesgos de seguridad propios de la colaboración entre agentes

La sesión menciona "seguridad empresarial" y "autenticación y autorización integradas" como característica de A2A, pero vale la pena precisar por qué importa más aquí que en un agente aislado:

- **Superficie de ataque ampliada:** cada *Agent Card* expuesta públicamente es, potencialmente, información de reconocimiento para un atacante (qué capacidades tiene el agente, qué esquemas de autenticación acepta).
- **Confianza transitiva:** si el *Client Agent* delega una tarea a un *Remote Agent* que a su vez delega a un tercero, la cadena de confianza (y de datos sensibles) puede extenderse más allá de lo que el diseñador original previó — el mismo riesgo de "sesgo algorítmico heredado" que ya aparecía en la ficha de RRHH de la Sección 2, pero ahora aplicado a una cadena de agentes en vez de a un solo modelo.
- **Aislamiento deliberado como mitigación:** el principio "agentes colaboran sin compartir memoria ni herramientas" no es solo una decisión de arquitectura limpia, es también una mitigación de seguridad: si un *Remote Agent* resulta comprometido, no tiene acceso directo al *Store* ni a las *tools* internas del *Client Agent*, solo a lo que la Task le entregó explícitamente.

6. Conexión con las sesiones anteriores

6.1 Con la Sesión 8 (arquitectura del agente)

Comunicación → Contexto → Entorno → Autonomía → Criticidad

Capa Sesión 8	Cómo se extiende en la Sesión 11
Communication Layer	Ya no es solo humano↔agente: ahora incluye agente↔agente (A2A)
Context Definition	El "contexto" de un agente puede incluir ahora la Agent Card de otro agente como fuente de capacidades disponibles
Environment Definition	El entorno de un agente puede incluir a otro agente como un tipo distinto de <i>tool</i> remota
Autonomy Dimension	Delegar una tarea a un agente externo (<i>blackbox</i>) es, en sí mismo, un acto de autonomía: el agente que delega no controla cómo se resuelve internamente
Criticality Dimension	La criticidad de un sistema multiagente depende también del agente más débil de la cadena, no solo del propio

6.2 Con la Sesión 9 (memoria contextual)

El contraste es directo y explícito en el propio material: mientras la Sesión 9 enseñó a **compartir** memoria entre turnos de un mismo agente (*Store*, *Checkpoint*), la Sesión 11 enseña el caso opuesto — agentes que **deliberadamente no comparten memoria entre sí**. El archivo plano (`reclamos_registrados.txt`) y, más adelante, la Task de A2A, cumplen el rol de "memoria compartida mínima": solo lo estrictamente necesario para la coordinación, nunca el estado interno completo de cada agente.

6.3 Con la Sesión 10 (tipos de agentes reflexivos)

Los dos agentes del laboratorio pueden clasificarse con la taxonomía de la Sesión 10:

Agente	Tipo (Sesión 10)	Por qué
<code>agente_reclamos_bancarios.py</code>	Model-Based Reflex	Mantiene estado (los datos del reclamo que va reuniendo turno a turno) y aplica reglas condición-acción sobre ese estado (pedir el siguiente dato faltante, o registrar si ya están todos)
<code>agente_analista_reclamos.py</code>	Goal-Based / con reglas explícitas	Su meta es "determinar una resolución final"; no simula estados futuros complejos, pero sí aplica una cadena de reglas de negocio condicionales para llegar a una decisión que cumple el objetivo del caso

La colaboración entre ambos no cambia el tipo individual de cada agente: un sistema multiagente puede combinar libremente agentes de distinto nivel de sofisticación (Sesión 10) coordinados por distintos medios de colaboración (Sesión 11).

7. Mapa del repositorio de la sesión

```
Sesion11_Agentes_Colaborativos/  
  SES11_M4_CollaborativeAgents.pdf  
  agente_reclamos_bancarios.py      ← Agente 1 (recepción, LangChain + archivo  
plano)  
  agente_analista_reclamos.py      ← Agente 2 (análisis + reporte .md,  
LangChain)  
  historial_chat.txt                ← Memoria de corto plazo del Agente 1  
(JSON por línea)  
  reclamos_registrados.txt          ← "Protocolo" de colaboración entre Agente  
1 y Agente 2  
  reportes_reclamos/  
    REC-F5898854.md                ← Reporte de resolución generado por el  
Agente 2  
  main.py                          ← Variante LangGraph (heredada de la  
Sesión 9, adaptada)  
  agente_langchain_reclamos.py      ← Variante create_agent + SqliteSaver +  
FileStore
```

7.1 Dependencias observadas en el código

```
langchain>=1.x      (create_agent, tool)  
langgraph>=1.x      (StateGraph, checkpoint.sqlite, store.memory) – solo en  
main.py / agente_langchain_reclamos.py  
langchain-anthropic  (model="anthropic:claude-sonnet-4-6")  
langchain-ollama     (model="ollama:llama3.2") – solo en main.py  
python-dotenv
```

8. Síntesis — qué se lleva el estudiante de esta sesión

1. **Colaborar no requiere un estándar:** dos agentes pueden coordinarse con un archivo de texto y un contrato de formato implícito. Es frágil, pero es el punto de partida correcto para entender *por qué* existe A2A.

2. **A2A formaliza tres cosas que el Bloque A resolvía de forma artesanal:** descubrimiento de capacidades (Agent Card en vez de "leer el código fuente del otro agente"), transporte estándar (HTTP + JSON-RPC en vez de un archivo compartido) y ciclo de vida de tareas (estados formales en vez de una columna ESTADO: de texto libre).
3. **A2A y MCP no compiten:** MCP conecta un agente con sus herramientas/datos; A2A conecta un agente con otro agente. Un sistema multiagente real casi siempre necesita ambos.
4. **El aislamiento entre agentes (no compartir memoria ni herramientas) es simultáneamente una decisión de diseño limpio y una mitigación de seguridad**, no solo una limitación técnica del protocolo.
5. La tarea del 25/07 pide aplicar este razonamiento al proyecto propio del estudiante: decidir si su sistema necesita más de un agente y, si es así, elegir primero el medio de colaboración más simple posible (archivos, variables) antes de justificar la complejidad adicional de A2A.